

Building a Ray Tracing Software

Salvador Rodriguez

Aug 1, 2024

1 Materials Needed

To start off this, ray tracing software doesn't need any extra software, besides your normal python libraries. I used only numpy and matplotlib.pyplot. As of hardware, you only need a computer and nothing else.

2 The Beginning

When thinking of ray tracing I thought that maybe with the time I had, I would be able to render something cool as a ray traced Lego Car, I envisioned it too. I didn't know what I was getting myself into, I only new some surface level things, but I didn't really know how it was supposed to be coded. Once I started looking at youtube videos and google searches of terms I didn't know I had a grasp of the idea behind ray tracing. The youtube videos had a lot of useful information but looking at the wikipedia page, it helped look at it from different angles. One important resource I want to point out is this book I found online called Ray Tracing in One Weekend by Peter Shirley, Trevor David Black, and Steve Hollasch. When starting to code my ray traced I realized that there are so so many things to pay attention to and the book and research I did made me scared and realized that there are so many ways to optimize this. I chose this project because I know it's a growing thing in today's technology side. I also like the different topics it includes, such as optics, random sampling, learning to code classes, subclasses, recursive functions, and how to use raster graphics.

3 The Project

The ray tracing software I made, sends out these rays from a camera into a scene. The components of this scene include of the scene, the viewport, the ray, and the camera. The camera just defines where you will be looking from and sending rays from. The viewport defines the screen that image will be displayed on. Light is usually as seen from emitting a light source and hitting an object and then our eye, creating an image. In ray tracing, we send rays through the eye into an object. In my software I didn't add any light sources because I don't have enough time and simple ray tracing just consists of using the background color. We shoot a ray, it hits an object and it bounce until it hits the background. The number of bounces determines how much light is absorbed, we combine the color of the background with the pixel we want to render via a recursive function. The more bounces it has, the darker pixel it is.

4 Results

In my code, you are able to change the resolution by just changing the image width, denoted by `img_w` in the camera class. You can also change the render scene, which you can just choose a sphere or a triangle to render. One feature I really like is the materials I can choose for the object, as of right now I have only shiny metal and lambertian material. Lambertian material uses lambertian distribution of diffused rays, which depict a nice matte-ish color. Another parameter you can change is the color of the object, choose any values between 0 and 1 for RGB. Also you can change the location and size of the objects. The sphere takes in the location vector and the radius as parameters, subsequently. The triangle object takes in three vertices. To get the exact ray traced image I have here, you just have to run `main.py`, the rest of the files in the folder are just classes that I use for main. Everytime you change a parameter, like size, material, or object, run `main.py`. To speed up time, just lower the resolution by going into camera class and changing `img_w`. To change color note that it's a parameter of the material. In the image below, I show different sized spheres and a triangle outside the vision of the triangle, but we can still see it's reflection. I hope to add more features and optimization, later on. Note this software isn't really user-friendly so it won't be asking what you want, you have to change it in the code itself.

